

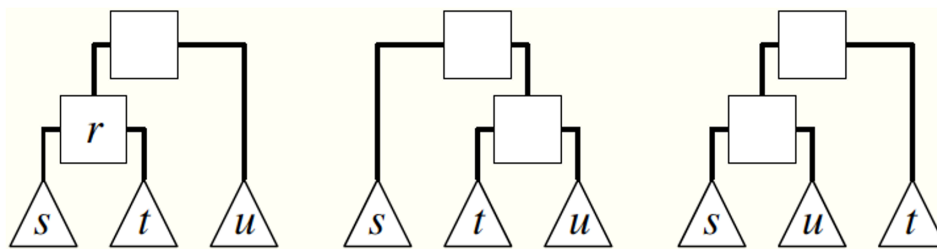
L12: How to Find the Optimal Dendrogram?

How can we determine the optimal dendrogram D from data?

Unfortunately, it is not possible to derive a closed-form solution for the optimal dendrogram, as we did for the optimal p_r values.

One option would be to construct an iterative search algorithm that samples random dendrograms and tries to find the dendrogram with the highest likelihood $\mathcal{L}(D)$. That may take too long, however, given the super-exponential number of possible dendrograms when n is large (see "food for thought" questions).

It would be much better to sample dendrograms in a biased manner with a probability that is proportional to the likelihood $\mathcal{L}(D)$ -- so that dendrograms with higher $\mathcal{L}(D)$ are more likely to be sampled by our search algorithm. We will see how to do so, using a stochastic optimization approach that is known as **Markov Chain Monte Carlo (MCMC)**.



Suppose that an iteration of the algorithm starts with a dendrogram D (e.g., the dendrogram shown at the left of the visualization). We pick an internal node r uniformly at random (other than the root). As shown in the visualization, r identifies three subtrees: the left child-subtree of r (called s), the right child-subtree of r (called t), and the sibling-subtree of r (called u). We can now create two more dendrograms by replacing either s or t with u (as shown in the visualization). Note that these two new dendrograms have preserved the internal relationships in the subtrees s , t and u . We now choose one of the two new dendrograms with equal probability -- let us call the new dendrogram D' .

The process we described above describes a *Markov Chain*, i.e., a probabilistic way to move from one dendrogram to the next, such that only the last dendrogram matters -- a Markov Chain does *not* remember its history.

Now that we have generated the new dendrogram D' , we calculate its likelihood $\mathcal{L}(D')$.

We accept D' as the new state of the Markov Chain if the likelihood has not decreased (i.e., if $\Delta\mathcal{L} = \mathcal{L}(D') - \mathcal{L}(D) \geq 0$).

If the likelihood has decreased, we may still accept D' but we do so with probability $e^{\log \Delta\mathcal{L}} = \frac{\mathcal{L}(D')}{\mathcal{L}(D)}$.

If we do *not* accept D' , then we remain with the dendrogram D and pick again a random internal node r .

This process of choosing whether to accept the new state D' or not is referred to as the Metropolis-Hastings rule.

The interesting property of the previous Markov Chain is that it is **ergodic** (i.e., we can go from any dendrogram to any other dendrogram with a finite series of the kind of transformation shown in the visualization).

Together with the use of the Metropolis-Hastings rule, this means that the MCMC algorithm presented here ensures that any dendrogram D will be sampled, at least asymptotically, with a probability that is proportional to the likelihood of that dendrogram $\mathcal{L}(D)$.

The algorithm terminates when the maximum observed likelihood has reached a "plateau" (meaning that we cannot improve $\mathcal{L}(D)$ over a number of iterations).

The previous process is stochastic, and so we may not get the same dendrogram each time we run the algorithm. In practice, however, it has been observed that all resulting dendrograms have roughly the same likelihood.

Food For Thought

a) Show that the number of possible dendrograms with n leaves is

$$n!(n-1)!/2^{n-1}.$$

b) If you are intrigued about MCMC, read more about Markov Chains, the ergodicity property, and the asymptotic sampling property mentioned above.